

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex realworld problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & DecisionMaking Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SCENARIO ASSESSMENT

Question 1: Navigation System – Farthest Pair of Delivery Points

Scenario

A logistics company uses a navigation system to plan delivery routes. Every day, hundreds or thousands of delivery locations are received as coordinates (x, y). Before planning the routes, the system must identify the two locations that are farthest apart. This information helps estimate the maximum travel distance and optimize vehicle allocation.

The current system uses the **brute-force approach**, which compares every possible pair of locations to find the maximum distance.

(a) Explain how the brute-force method works

The brute-force algorithm is the simplest way to solve the problem. It does not try to optimize the search. Instead, it checks every possible pair of delivery points.

Working Steps

1. Read all delivery point coordinates.
2. Select the first point.
3. Compare it with every remaining point.
4. Calculate the distance between the two points.
5. Store the largest distance found.
6. Repeat the process for every point.
7. Return the pair having the maximum distance.

Since every possible pair is checked, the algorithm always gives the correct answer.

Distance Formula

For two points

$$P_1(y_1), P_2(y_2)$$

Distance is

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

The algorithm computes this distance for every pair.

Example

Delivery Points

Point Coordinates

A (2,3)

B (5,7)

C (8,2)

D (1,9)

The comparisons are

- A-B
- A-C
- A-D
- B-C
- B-D
- C-D

The largest distance among these pairs becomes the answer.

Pseudocode

Read n points

maxDistance = 0

For i = 1 to n-1

 For j = i+1 to n

 distance = CalculateDistance(i,j)

 If distance > maxDistance

 maxDistance = distance

 Store pair

Display pair

(b) Derive the time complexity

Suppose there are **n** delivery points.

The first point is compared with

$$n - 1$$

points.

The second point is compared with

$$n - 2$$

points.

The process continues until the last point.

Total comparisons

$$(n - 1) + (n - 2) + (n - 3) + \dots + 1$$

Using the arithmetic series formula,

$$= \frac{n(n-1)}{2}$$

Each comparison requires one distance calculation

$$T(n) = \frac{n(n-1)}{2}$$

Ignoring constants,

$$O(n^2)$$

Space Complexity

Only a few variables are required.

Therefore,

$$O(1)$$

excluding the input coordinates.

(c) Why does it become inefficient?

The brute-force algorithm checks every possible pair.

When the number of locations becomes very large, comparisons increase rapidly.

Example

Points Comparisons

10 45

100 4950

1000 499500

10000 49995000

Nearly **50 million** distance calculations are required for only 10,000 locations.

Problems

- High execution time
- High CPU usage
- Increased battery consumption
- Poor scalability
- Not suitable for real-time navigation

Advantages

- Very easy to understand
- Simple implementation
- Always produces the correct answer
- Works for any number of points

Disadvantages

- Slow for large datasets
- Performs unnecessary comparisons
- Cannot handle real-time applications efficiently
- Poor scalability

Real-world Applications

- Delivery route planning
- Drone navigation
- Geographic Information Systems (GIS)
- Emergency vehicle planning
- Warehouse logistics

Question 2: Document Analysis System – Brute Force String Matching

Scenario

A document analysis system scans books, reports, research papers, and legal documents to find specific keywords.

For example, it may search for

- Artificial Intelligence
- Machine Learning
- Confidential
- Password

The current system uses **Brute Force String Matching**.

(a) Explain how the algorithm works

The algorithm searches for a pattern inside a text by checking every possible position.

If the text length is **n** and pattern length is **m**, the algorithm starts from the first character of the text.

If characters match, comparison continues.

If a mismatch occurs, the pattern shifts one position to the right.

This continues until the end of the text.

Working Steps

1. Read the text.
2. Read the keyword.
3. Start comparison from the first character.

4. Compare characters one by one.
5. If mismatch occurs, move one position.
6. Repeat until the keyword is found or the text ends.

Example

Text

DATA STRUCTURES AND ALGORITHMS

Pattern

STRUCT

The algorithm checks

DATA...

Mismatch

Shift one position

ATA...

Mismatch

Shift again

Continue until

STRUCT

Match found.

Pseudocode

For $i = 0$ to $n-m$

$j = 0$

 While $j < m$

Compare text[i+j]

Compare pattern[j]

If mismatch

break

j++

If j == m

Pattern Found

(b) Analyze the time complexity

Suppose

Text length

n

Pattern length

m

Worst case

The algorithm compares all **m** characters at every position.

Total positions

$$n - m + 1$$

Worst-case complexity

$$O(nm)$$

Best Case

Every first character mismatches immediately.

Time Complexity

$$O(n)$$

Average Case

Generally lies between best and worst case.

Space Complexity

Only a few variables are used.

Therefore,

$$O(1)$$

(c) Inefficiency for large inputs

Suppose

Document size

100 MB

Keyword

Artificial Intelligence

The brute-force algorithm repeatedly compares the same characters.

As document size increases,

- Execution time increases.
- CPU utilization increases.
- Memory access increases.
- Searching multiple keywords becomes very slow.

Comparison with Efficient Algorithms

Algorithm	Time Complexity	Main Idea
-----------	-----------------	-----------

Brute Force	$O(n \times m)$	Checks every position
KMP	$O(n + m)$	Uses prefix table to avoid rechecking
Rabin–Karp	Average $O(n + m)$	Uses hashing for faster matching
Boyer–Moo re	Often sublinear	Compares from right to left and skips characters

Advantages

- Simple to implement
- No preprocessing required
- Works for all patterns
- Easy to understand

Disadvantages

- Slow for large documents
- Repeats many comparisons
- Poor performance for long patterns
- Not suitable for search engines or large databases

Real-world Applications

- Keyword searching in documents
- Plagiarism detection (basic implementations)
- Text editors (simple Find feature)
- Log file searching

- Educational tools for learning string matching

Overall Conclusion

Brute-force algorithms are easy to understand and guarantee correct results because they examine all possible solutions. However, they become inefficient as the input size increases due to their high time complexity. In the navigation problem, the algorithm performs $O(n^2)$ distance calculations, while in string matching it requires $O(n \times m)$ comparisons in the worst case. For large datasets and real-time applications, optimized algorithms such as **Convex Hull methods** (for geometric problems) and **KMP, Boyer–Moore, or Rabin–Karp** (for string searching) provide much better performance.